The Narrow Reach of LLMs into Program Option Spaces: A Case for Configuration Manifests

Anonymous Author(s)
Submitted for double-anonymous review

***Abstract*—Command-line programs can document hundreds of options, and large language models (LLMs) now write many of the scripts that call them. LLMs have likely seen both the documentation of these programs, which lists every option, and code that calls them, which uses only a few, but it is unclear how far LLMs reach into these option spaces and which source guides them. We mine public repositories for 31,017 invocations of 403 programs and subcommands from GNU utilities, Git, and CI tooling, in scripts and workflows attributed to LLMs or written by developers, and in the Linux kernel build system. In each dataset, 90% to 98% of the documented options are never used in these artifacts. Scripts attributed to LLMs reach 5.5% of the GNU options, about as many as developer scripts (5.4%) and fewer than kernel scripts (7.0%). Both rank shared options in a moderately similar order (Spearman $\rho \approx 0.6$), but options used only by LLMs are mostly general purpose, whereas those used only by developers tend to serve a specific task. Two LLM agents resolving GitHub issues reach at most 1.4% in 16,172 executed invocations, mostly with options their prompts name or demonstrate. This is consistent with LLMs following how options are used rather than documented, which leaves rare or recent options, and those of internal or new programs, out of reach unless an LLM is given their documentation. We therefore propose *configuration manifests*, in which programs describe their options and constraints for LLMs to query, and *contracts* that bind a manifest to a program version and an implementation.**

***Index Terms*—option spaces, configuration spaces, large language models, command-line programs, empirical software engineering, software configurability, mining software repositories**

I. Introduction

Almost all modern software systems are configurable, and the valid combinations of their configuration options (*a.k.a.* features) form their configuration space [1], [2], [3], [4]. Depending on the system, these options can be set in different ways, such as in configuration files, through environment variables, or in graphical interfaces. Programs that run on the command line are configured mainly through the options passed with each invocation, such as `-l` to list files in long format with the `ls` program, and a single such program can document hundreds of configuration options [5]. Invocations of such programs, with or without options, are often used in shell scripts, Dockerfiles, and continuous integration (CI) workflows, which software projects keep in version control along with their source code. The configuration options that developers choose for a command-line program can therefore be observed at scale in software repositories.

For a long time, developers wrote such scripts and workflows themselves. Today, large language models (LLMs) write a growing part of them [6], [7], [8], and LLM agents also run program invocations directly in a shell [9], [10]. For every invocation, inside a script or in a shell, the LLM decides which options of the program to pass, and these options determine what the program does. Listing 1 shows how an LLM (Claude Opus) answered our prompt to create a release archive of a source tree. The command is valid and uses 3 of the 215 options that `tar` documents. With further options of `tar`, however, such as `--sort=name` and a fixed `--mtime`, the archive could have been made reproducible byte for byte [11]. One may argue that the prompt did not ask for a reproducible archive. Still, in the prompts we give to LLMs, not every requirement of a task is always stated, and in the `tar` case, many projects expect their release archives to be reproducible.

```
# Prompt: "Create a release archive of the source tree."
$ tar -czf release.tar.gz src/    # LLM answer

# Reproducible variant the LLM could also generate:
$ tar --sort=name --mtime=@0 --owner=0 --group=0 \
      --numeric-owner --format=gnu -czf release.tar.gz src/
```

Listing 1. Command generated by an LLM and a reproducible variant.

Missing options like these are easy to overlook. Code generated by LLMs is usually evaluated for functional correctness, with tests [12] or real repository issues [13], and a command can pass these checks while it omits options that would make it reproducible, robust, or safe. *Yet, little is known about how many of the documented program options LLMs actually use, which ones they choose, and how this compares with the options that developers use.* This matters beyond single commands. If LLMs imitate existing usage rather than documentation, then rarely used or recently added options, and the options of internal or new programs that no public code calls, are unlikely to be chosen by an LLM.

In this paper, we present an empirical study that measures how many of the documented options of programs appear in the invocations that LLMs write and run, and compares these options with those that developers use. We organise the study around four research questions: *how much of the documented option set of a program is used in practice* (RQ1), *whether LLMs use the same options as developers* (RQ2), *whether LLMs and developers use the options they share in the same way* (RQ3), and *whether LLM agents that execute commands use more options* (RQ4). To answer these questions, we recover the documented options of 403 programs and subcommands from GNU utilities, Git, and CI tooling, 9,604

options in total, from their `--help` output. We then mine 851 public repositories and extract the options used in about 31,000 invocations of these programs from 976 scripts and CI workflows attributed to LLMs, 779 written by developers, and 1,081 build and automation scripts of the Linux kernel. We also analyse 16,172 invocations of the GNU utilities that two LLM agents executed in 1,200 runs.

Our results show that about nine in ten documented program options are never used. Scripts attributed to LLMs reach about as many options as scripts written by developers, 5.5% versus 5.4% of the GNU options, while the kernel scripts reach 7.0%. On CI tooling, both reach about 1.1%, and on Git, LLMs reach 2.7% versus 1.9%, with more than twice as many invocations. LLMs and developers rank shared options with a moderate correlation and mostly spell them alike, but they share on average only 42% of the options that each uses. Program options used only by developers tend to serve a specific task, whereas those used only by LLMs tend to be general purpose. The two agents reach at most 1.4% of the GNU options, mostly the ones named in their prompts. These results are more consistent with *LLMs drawing on usage than on documentation*, which motivates our proposal that programs describe their options in a form that LLMs can query.

In summary, this paper makes the following contributions:

- A ground truth of the documented option sets of 403 command-line programs and subcommands (Section III-D).
- An empirical study that compares, with four metrics, the options used by LLMs, developers, and the Linux kernel scripts in three datasets, complemented by the commands of two LLM agents (Sections III and IV).
- A proposal of *configuration manifests* and versioned *configuration contracts* that make the option sets of programs queryable by LLMs (Section VI).
- A replication package with all scripts, corpora, ground truth files, and analysis notebooks[1].

Sections II and III give the background, motivation, and describe the study design. Section IV answers the research questions, Sections V and VI discusse their implications and presents our proposal. Sections VII and VIII cover threats to validity and related work, and Section IX concludes the paper.

II. Background and Motivation

A. Options and Configuration Spaces

A command-line program exposes its runtime variability through *options*, that is, arguments that start with one or two dashes and that a caller passes after the program name. For example, `rm -f` suppresses confirmation prompts and `grep -r` searches directories recursively. Many options have a short and a long spelling, such as `-r` and `--recursive`. We call the set of all options that a program documents its *documented option set*. In software product line terminology [2], options correspond to features, and the valid combinations of features, together with the constraints among them, form the *features configuration space* [1], [3]. The documented configuration option set is thus a flat list, whereas the configuration space also includes the combinations and their constraints. Without any option, a program runs with its *default behaviour* [14], and each option a caller adds changes that behaviour.

B. Choosing Options

Knowing the available documented configuration options of a program is only a first step towards configuring it well. First, the options of a program interact, and many of the constraints among them are not stated in its documentation. Research on configurable systems shows that such constraints usually have to be recovered through dedicated analyses [15], [16]. Second, the right option for one program often depends on the other programs in a pipeline, for example on the output format of the preceding command. Third, the option set of a program changes over time and differs between implementations. Even small GNU programs gain variability across releases, and programs with the same name, such as the `ls` of GNU coreutils and that of BusyBox, accept different options [17]. Finally, the caller has to know both that an option exists in a given program and that the task at hand needs it.

Moreover, misconfiguration has long been recognised as a common source of failures. Empirical studies attribute a large share of production incidents to misconfiguration rather than to code defects [18], [19], [20]. For command-line programs, the consequences are concrete. For instance, a CI step that runs a test suite without a timeout can block the pipeline when a test hangs, and a `docker build` without `--pull` may reuse an outdated base image that is cached on the build machine.

C. LLMs as Callers of Command-Line Programs

LLMs take part in writing program invocations in two ways. As *coding assistants*, they generate scripts and workflow files that developers review, or just accept, and commit. As *agents*, LLMs run commands themselves. The agent emits an invocation, a sandbox executes it, and the output is returned to the model as an observation [9], [10], a capability that widely used assistants now offer [21]. In both settings, it is the LLM, not a developer, that chooses the program options.

D. Motivation

An LLM can learn about the options of a program from two sources. The documentation, which lists all options of a program, and for a program as common as `tar` it is very likely part of the training data. Anthropic, for example, states that Claude *"knows common utilities like unix tools"* [21]. Or, code that calls a program shows which options are used together and for what purpose, but only for the small part of its options that callers actually use [5]. The two sources lead to different expectations. An LLM that draws on the documentation should reach well beyond the options that developers use. An LLM that draws mainly on usage should stay within the frequent part of developer usage, since LLMs are less reliable on content that is rare in their training data [22], [23]. Such popularity effects have been reported for the configuration parameters

[1]Anonymised replication package, https://github.com/anonconfs/llms-miss.

that LLMs validate [24], the libraries they choose [25], and the APIs they call correctly [26], and related effects for insecure code patterns [27] and package names [28]. On the other hand, studies show that documentation given in the prompt helps LLMs use APIs and commands that are rare in their training data [26], [29]. To our knowledge, no study has measured which of the two sources prevails for program options.

This question is not limited to popular programs. Documentation rarely states which options require or exclude each other (Section II-B), and Anthropic reports that even the schemas written for LLM tools cannot express *"when to include optional parameters"* or *"which combinations make sense"* [30]. The option sets of programs also change across releases [17], while the knowledge of an LLM is fixed at training time, and LLMs have been shown to keep using deprecated APIs [31]. For a new option, a new program, or an internal tool, no public usage exists at all, and an LLM can only rely on the documentation it is given [21], [32]. Popular programs are the most favourable case, since both their documentation and their usage are well represented in training data. If LLMs follow usage even for these programs, the other cases are likely to be harder. Finally, unlike the habits of developers, the behaviour of an LLM can be changed centrally, through prompts, training, or the documentation that an agent may query. Knowing where LLMs fall short in their use of program options shows where such changes could help. The answer also concerns the programs themselves, as options that no caller uses are either missed capabilities or unnecessary variability that adds code and maintenance effort [5], [33].

Comparing LLMs with developers. Observing that an LLM uses few options says little on its own, because a single developer also uses only a small part of the options of a program [5]. We therefore decided to compare LLMs with developers on the same programs. Developers are the natural baseline, since the programs and their documentation were designed for them, and the comparison separates limitations that are specific to LLMs from limitations shared by all callers. Note that we measure *configuration diversity*, that is, which documented options are used, and not *configuration adequacy*, that is, whether the options used are right for the task. Using more options is not better in itself. Many tasks need only a few options, and an option that only some implementations of a program accept makes a script less portable (Section II-B).

III. Study Design

A. Goal and Research Questions

Following the Goal Question Metric approach [34], we *analyse* program invocations written by LLMs and by developers *for the purpose of* comparing the documented options they use, *with respect to* which options are used and how, *from the viewpoint of* researchers and builders of development tools that rely on LLMs, *in the context of* GNU programs, Git, and CI tooling. We derive four research questions.

- RQ1 **(Saturation). How much of a program's documented option set is used in practice?**
- RQ2 **(Divergence). How does the part of a program's option set used by LLMs compare with the part used by developers?**
- RQ3 **(Style). When LLMs and developers use the same option, do they use it in the same way?**
- RQ4 **(Execution). Do LLM agents that execute commands and observe their output use a wider part of the option sets of programs?**

RQ2 uses the kernel corpus as an additional reference for the GNU programs, and RQ4 checks whether the results for static artifacts also hold when an LLM runs the commands itself.

B. Definitions

A *unit* is the entity that owns a configuration option set. For instance, for the GNU programs, a unit is a single program (*e.g.*, `tar`). For Git, it is a subcommand (*e.g.*, `git commit`). For CI tooling, it is a program together with its subcommand (*e.g.*, `docker build` or `npm install`).

An *invocation* is a single call of a unit with its configuration options. An artifact, such as a shell script or a CI workflow, usually contains many invocations, so the invocation, not the file, is our unit of analysis.

A *caller* is the author of an invocation, that is, a developer who writes a script or an LLM that generates it.

A *corpus* is a set of artifacts with a common provenance. For each dataset, we study a developer corpus and an LLM corpus, and for the GNU programs also a kernel corpus. We describe the artifacts of the LLM corpora as *attributed* to an LLM, because their authors state, through a provenance marker, that an LLM generated them. The term reflects this declared origin. Although we include only artifacts with an explicit marker (Section III-E), the markers give no evidence of whether developers adjusted an artifact before committing it, so we cannot exclude such adjustments (Section VII).

C. Datasets

To answer the research questions, we compute four metrics, namely reach, concentration, partition, and style (Section III-F), on three datasets that differ in domain, size, and type of artifact (Table I), and we use the trajectories of two LLM agents for RQ4.

Dataset A: GNU and Related Programs: This dataset covers 118 programs from GNU coreutils and from packages that are commonly installed alongside it, such as `tar`, `grep`, `find`, `gawk`, `sed`, and `diff`, which together document 2,716 options. The developer corpus consists of 293 shell scripts from 25 popular open source repositories on GitHub, such as systemd, curl, the Docker library images, ohmyzsh, and pyenv, all committed before 2020. The LLM corpus consists of 298 scripts from public repositories whose provenance markers name ChatGPT (157 scripts), Claude (93), or GPT (30), or state AI generation without naming a model (18). The kernel corpus consists of 1,081 build and automation scripts from the Linux kernel source tree. We use it as a reference from a single project with a long history, whose scripts are reviewed by maintainers and call many different system

TABLE I
SUMMARY OF THE THREE DATASETS.

	A. GNU	B. CI	C. Git
Units (programs/subcommands)	118	135	150
Documented options	2,716	1,864	5,024
Files, developer	293	210	276
Files, LLM	298	210	468
Files, kernel	1,081	—	—
Repositories, developer	25	28	48
Repositories, LLM	264	180	364
Corpus size (MB)	8.8	2.6	10.6
Invocations, developer	4,837	96	564
Invocations, LLM	8,795	186	1,268
Invocations, kernel	15,271	—	—

utilities. Its tasks, however, differ from those of the other two corpora, which we take into account when interpreting the results. Together, the three corpora contain about 28,900 invocations of the 118 units.

Dataset B: CI Tooling: This dataset covers the command-line tools that dominate CI pipelines [35], namely Docker (with Docker Compose), npm, and pip, with 135 units and 1,864 documented options. The developer corpus contains 210 GitHub Actions and other CI workflow files from 28 large open source projects, such as TensorFlow, PyTorch, Node.js, Django, and Angular, all committed before late 2022. The LLM corpus contains 210 CI workflow files from 180 public repositories with a provenance marker stating AI generation. With 282 invocations, this is our smallest dataset, and we interpret its results with care. We include it because CI pipelines run on every change, so a missing option can lead to failed builds or wasted compute.

Dataset C: Git: This dataset covers all 150 Git subcommands, which document 5,024 options. The developer corpus consists of 276 files from open source projects, of which 215 are CI workflow files and the rest shell scripts. The LLM corpus consists of 468 files attributed to LLMs, namely 296 workflow files and 172 shell scripts. Git complements the GNU programs because of its two levels, as a caller first chooses a subcommand and then its options.

Agent Trajectories: For RQ4, we use two public sets of agent trajectories on GitHub issues, namely 600 runs of SWE-agent [9] on 35 issues, 570 with Llama 70B and 30 with Llama 8B [36], and 600 runs of DeepSWE-Agent with Kimi K2 on 595 issues [37]. We refer to the two agents by their main models, Llama 70B and Kimi K2, and count the 30 runs with Llama 8B under Llama 70B. In each run, a sandbox executes the shell commands of the agent. We extract every executed invocation of the 118 units of Dataset A, which gives 1,802 invocations for Llama 70B and 14,370 for Kimi K2.

D. Ground Truth Recovery

For every unit, we recover the documented option set from the installed program by running its help command (`--help` or the help of the subcommand). We used GNU coreutils 9.10, Git 2.54, Docker 29.4, npm 10.9, and pip 26.0, and the replication package lists the versions of all other packages. A parser based on regular expressions extracts short options (`-x`) and long options (`--word`) from the help text and records each spelling as a separate entry. The result is one ground truth file per unit and a summary with the number of short, long, and total options. In total, we recovered 9,604 documented options for 403 units (*cf.* first two rows in Table I).

For two program families, the parser needed extensions. For Git, no single help output is complete. The `-h` output of several subcommands is only a usage line, some documented synonyms, such as `--staged` of `git diff`, appear only in the prose of the manual page, and bracketed forms such as `--[no-]verify` combine two options in one token. We therefore take the union of three sources per subcommand, namely the `-h` output, the option lines of the manual page, and the options listed by Git's completion helper, and we expand bracketed forms into both spellings. For `find` and `stty`, which do not follow the GNU option conventions, we also extract options that are words with a single dash (*e.g.*, `-name`).

E. Invocation Extraction

We extract every invocation of a target unit from each artifact, *e.g.*,, `tar -czf out.tgz src/` in a shell script is an invocation of the unit `tar`. The extraction has two steps.

Step 1 (locating invocations): For the shell scripts of Dataset A, we split each line into commands at pipes, semicolons, `&&`, and `||`, and we also process the commands inside command substitutions. Quoted strings are skipped, common prefixes such as `sudo`, `env`, `nohup`, and `xargs` are removed, and so is a path before the program name (*e.g.*, `/usr/bin/sort` becomes `sort`). If the first word of a command is a target unit, we keep it together with all following tokens up to the end of the command. For the kernel corpus, we apply the same parser to the Makefile recipes and the embedded shell fragments. For the CI workflows of Dataset B, a simpler extractor selects the lines that start with a call of `docker`, `docker compose`, `npm`, or `pip`, mostly in the `run` steps of a workflow, takes the word after the program as the subcommand, and collects the tokens of the line that look like options. For Dataset C, the extractor records each line that calls a Git subcommand with at least one option.

Step 2 (normalising options): Each token that starts with a dash is a candidate option, and all other tokens are arguments. We strip `=value` suffixes (*e.g.*, `--sort=name` becomes `--sort`). For Dataset A, we also split bundled short options when all their letters are documented short options of the unit (*e.g.*, `-czf` becomes `-c`, `-z`, and `-f`), keep only the option letter when a value is attached (*e.g.*, `-n1` becomes `-n`), and skip numbers, such as the `-5` of `head -5`, the `--` that marks the end of the options, and test operators such as `-eq`. A candidate counts as an option only if it matches an entry in the ground truth of the unit. On GNU and Git, between 98.2% and 100% of the candidates in each corpus match. On CI tooling, only 42.6% (developer) and 63.6% (LLM) do, mostly because the extractor neither expands short forms

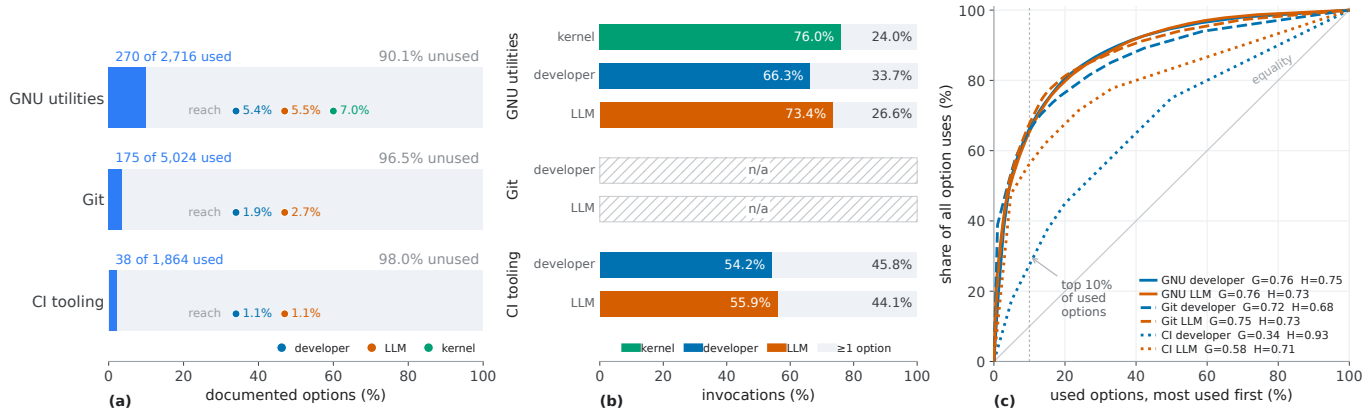


Fig. 1. Use of the documented option sets (RQ1). (a) Options used by any corpus and reach per corpus. (b) Invocations with and without options (n/a for Git, where invocations without options are not recorded). (c) Concentration (Gini G, normalised entropy H).

nor skips global options and thus cannot identify the called unit, as for `npm i`, a short form of `npm install`, or for `docker-compose -f file up`, where a global option precedes the subcommand, and because options of an inner command, such as the `-c` of `sh -c` inside `docker exec`, are attributed to the outer one. We keep an invocation without options as a record with zero options, so that the number of invocations and the number of options per invocation come from the same table. The only exception is Git, for which the extractor records only invocations with at least one option.

Aliases: Many options have a short and a long spelling, *e.g.*, `-r` and `--recursive` of `cp`. We identify such pairs from the lines of `--help` that list both spellings. For reach and partition (RQ1 and RQ2), we count each spelling as a separate option, and for style (RQ3), we compare which spelling each corpus prefers. Options with the same name in different units we count separately, *e.g.*, `ls -l` and `wc -l`.

Temporal boundary and provenance: To keep the artifacts of developers apart from those generated by LLMs, the GNU developer corpus and the kernel corpus contain only scripts committed before January 2020, that is, before GPT-3 was released in June 2020. The CI developer corpus contains only workflows committed before the public release of ChatGPT in November 2022. We identified the artifacts attributed to LLMs through explicit provenance markers in commit messages, file headers, or README files, which state that the artifact was generated with an LLM and, for most GNU scripts, name the model. Such markers have also been used to study how developers acknowledge their use of generative AI in open source projects [38]. All analysis steps are deterministic and implemented as Bash scripts and Python notebooks.

F. Metrics

We use four metrics: *reach* measures what share of the documented options a corpus uses, *concentration* how evenly its uses spread over them, *partition* which options the corpora share, and *style* how they use the shared options. We compute each metric in the same way for the developer corpus, the LLM corpus, and, for Dataset A, the kernel corpus and the two agents. As explained in Section II-D, all metrics describe configuration diversity, not configuration adequacy.

Reach (RQ1, RQ2, RQ4): Reach is the share of the documented configuration options that a corpus (*i.e.*, developer, LLM, or kernel) uses. Let D be the set of options documented for all units, that is, the ground truth, and let $U \subseteq D$ be the set of distinct options used in a corpus. Reach is the ratio

$$R = |U|/|D|, \text{ and } 1 - R$$

is the share of documented options that the corpus never uses.

Concentration (RQ1, RQ4): Reach counts the options a corpus uses but not how evenly the uses are distributed over them. Concentration captures whether a few options dominate the usage in a corpus. Let $x_1, \ldots, x_k$ be the usage counts of the k options a corpus uses, with mean $\bar{x}$ and shares $p_i = x_i / \sum_j x_j$. We report two measures. The Gini coefficient [39],

$$G = \frac{\sum_i \sum_j |x_i - x_j|}{2k^2 \bar{x}} \in [0, 1],$$

is 0 when all used options are used equally often and approaches 1 when the uses concentrate on a single option. The normalised Shannon entropy [40],

$$H = -\frac{1}{\ln k} \sum_i p_i \ln p_i \in [0, 1],$$

measures the same imbalance in the opposite direction and decreases towards 0 as the uses concentrate on fewer options.

Partition (RQ2): Partition compares the developer and LLM corpora by splitting the programs options they use into three disjoint subsets, namely options used by both (*shared*), only by developers (*developer only*), and only by LLMs (*LLM only*). For Dataset A, we also report the options that only the kernel corpus uses (*kernel only*), as a fourth subset.

Style (RQ3): Style compares how the developer and LLM corpora use the options they share, in three respects. *Popularity* is the Spearman rank correlation [41] between the usage counts of the shared options in the two corpora. *Spelling* is the share

of uses in short rather than long form, restricted to the alias pairs of Section III-E, so that a choice between forms exists, and a corpus prefers the short form of an option if at least half of its uses are short. *Configuration depth* is the distribution of the number of options per invocation.

IV. Results

We report the results for each research question in turn.

A. RQ1: How Much of the Option Set Is Used?

Figure 1(a) shows, for each dataset, the share of documented options that appear in at least one corpus. For the GNU programs, only 270 of the 2,716 options are used, so 90.1% are never used. For Git and CI tooling, the unused share is even higher, at 96.5% and 98.0%. The kernel corpus has the highest reach, with 7.0% of the GNU options, while the developer and LLM corpora reach 5.4% and 5.5% on GNU and less than 3% on Git and CI tooling.

Larger corpora would likely contain more options. However, the observed usage is both shallow and concentrated, which suggests that most documented options would remain unused. As Figure 1(b) shows, between 66.3% and 76.0% of the GNU invocations pass no option, *i.e.*, they run the program with its default behaviour. When options are used, most uses fall on a few of them. The Gini coefficient of the option usage counts is 0.76 for both GNU corpora (Figure 1(c)), and the top 10% of the used options account for about two thirds of all uses. The normalised entropy is 0.75 for developers and 0.73 for LLMs on GNU, well below the maximum of 1.0. On average, a GNU invocation carries 0.45 options in the developer corpus and 0.34 in the LLM corpus. The CI developer corpus is an exception, since its uses are spread more evenly ($G = 0.34$, $H = 0.93$), but its reach is just as low (1.1%, Figure 1(a)).

Finding 1. Across the three datasets, between 90.1% and 98.0% of the documented program options are never used. The uses concentrate on a few options (Gini between 0.72 and 0.76 on GNU and Git), and 66% to 76% of the GNU invocations pass no option. Like the developer corpora, the LLM corpora use only a small share of the option sets (1.1% to 5.5%), although not the same options (RQ2).

B. RQ2: Do LLMs Use the Same Options as Developers?

To compare the parts of the option sets that LLMs and developers use, we first examine in detail how many options each corpus uses (Figure 2(a)), and then how many of these options the corpora share (Figure 2(b)).

We observed that on GNU, the LLM corpus uses 5.45% of the options (148) and the developer corpus 5.38% (146), although the LLM corpus contains almost twice as many invocations. The kernel corpus reaches 7.03% (191). On CI tooling, the two corpora reach almost the same share, 1.13% (21 options) for LLMs and 1.07% (20) for developers. On Git, the LLM corpus reaches more, 2.73% (137) against 1.93% (97), but it also has more than twice as many invocations (1,268 against 564), which may partly explain its higher reach.

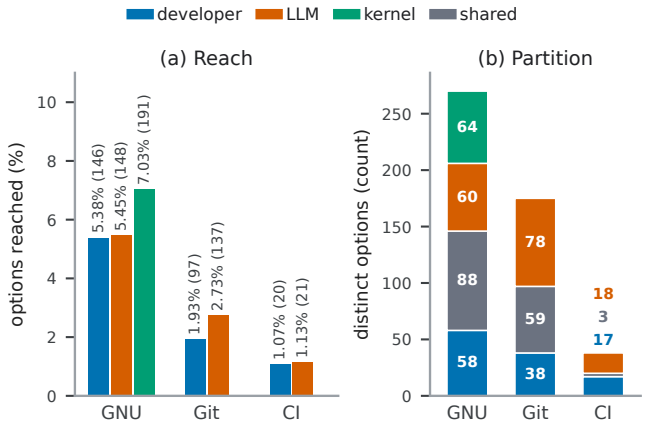


Fig. 2. Options used by each corpus (RQ2). (a) Reach, with the number of options in parentheses. (b) Partition into shared and exclusive options.

Parts of similar size can still contain different options. Figure 2(b) splits the options used by developers and LLMs into a shared part and two exclusive parts, and Table II lists the ten most used options of each part. On GNU, 88 options are shared, while 58 are used only by developers and 60 only by LLMs, so the two corpora share about 60% of the options that each uses. On Git, they share 59 options, 61% of the developer options but only 43% of the LLM options, as LLMs use 78 options that developers never use. On CI tooling, they share only 3 options, 15% of the developer options and 14% of the LLM options. The kernel corpus adds 64 GNU options that neither of the other two uses, such as `mktemp -u`, `md5sum -c`, and `stdbuf -o`.

The exclusive GNU options also differ in kind. Table II shows that many of the options that only developers use serve a specific task, such as selecting files by owner (`find -user`), installing a file with a given owner or into a new directory (`install -o`, `install -D`), or comparing two files silently in a condition (`cmp -s`), whereas many of the options that only LLMs use are general purpose, such as excluding files from an archive (`tar --exclude`), searching recursively (`grep -r`), or reporting disk usage (`du -s`, `du -h`). LLMs thus cover the options that general tasks need, but rarely those that a script needs for a specific job.

Finding 2. LLMs and developers use parts of the option sets of similar size (5.45% vs. 5.38% of the GNU options), except on Git, where LLMs reach more (2.73% vs. 1.93%). Yet they share only part of these options, on average 42% of those that each uses (60% on GNU, 52% on Git, and 15% on CI). Options that only developers use tend to serve specific tasks, whereas those that only LLMs use tend to be general purpose.

C. RQ3: Do LLMs and Developers Use Shared Options Alike?

Figure 3(a) plots each *shared option* by its share of developer uses and its share of LLM uses. The points follow the

TABLE II

THE TEN MOST USED OPTIONS IN EACH PART OF THE PARTITION (FIGURE 2(B), RQ2), WITH THEIR NUMBER OF USES (DEVELOPER/LLM FOR SHARED OPTIONS). TIES AT THE TENTH RANK ARE BROKEN ALPHABETICALLY, AND GIT SUBCOMMANDS ARE SHOWN WITHOUT THE `GIT` PREFIX.

GNU Shared		Developer only		LLM only	
`echo -e`	189/671	`find -user`	31	`tar --exclude`	22
`rm -f`	180/182	`install -D`	17	`find -path`	19
`cut -f`	232/82	`cmp -s`	14	`grep -r`	17
`mkdir -p`	113/200	`install -T`	11	`du -s`	11
`cut -d`	217/63	`gunzip -c`	10	`grep -h`	11
`grep -q`	37/165	`sed -f`	9	`tar -v`	11
`grep -E`	19/137	`grep -P`	8	`grep -I`	10
`tr -d`	89/51	`install -o`	8	`du -h`	8
`sed -e`	106/11	`cp -v`	6	`date -f`	7
`rm -r`	50/52	`mktemp -t`	6	`grep --include`	6

Git Shared		Developer only		LLM only	
`config --global`	259/272	`config --bool`	10	`push -u`	22
`commit -m`	25/158	`diff --color`	6	`branch --show-current`	18
`diff --name-only`	27/88	`fetch --quiet`	5	`status --short`	12
`diff --quiet`	13/102	`commit -v`	4	`tag -a`	9
`diff --cached`	5/107	`switch -c`	4	`tag --sort`	8
`checkout -b`	12/56	`checkout --theirs`	3	`branch -D`	5
`config --local`	27/25	`log --name-status`	3	`checkout -B`	5
`status --porcelain`	15/33	`pull --no-rebase`	3	`cherry-pick --continue`	5
`add -A`	4/40	`show --no-patch`	3	`config -f`	5
`diff --staged`	4/35	`commit -a`	2	`log --grep`	5

CI Shared		Developer only		LLM only	
`pip install -r`	3/4	`docker exec -t`	7	`npm install -g`	30
`docker run --rm`	4/1	`docker run -v`	4	`npm audit --audit-level`	5
`docker run --name`	1/1	`docker build --build-arg`	2	`npm audit --json`	3
		`docker build -f`	2	`npm install --save-dev`	3
		`docker build -t`	2	`npm install --package-lock-only`	2
		`docker ps --all`	2	`npm outdated --json`	2
		`docker run --interactive`	2	`docker run -d`	1
		`docker run -c`	2	`docker-compose down --remove-orphans`	1
		`docker exec -w`	1	`docker-compose down -v`	1
		`docker rm -f`	1	`docker-compose up -d`	1

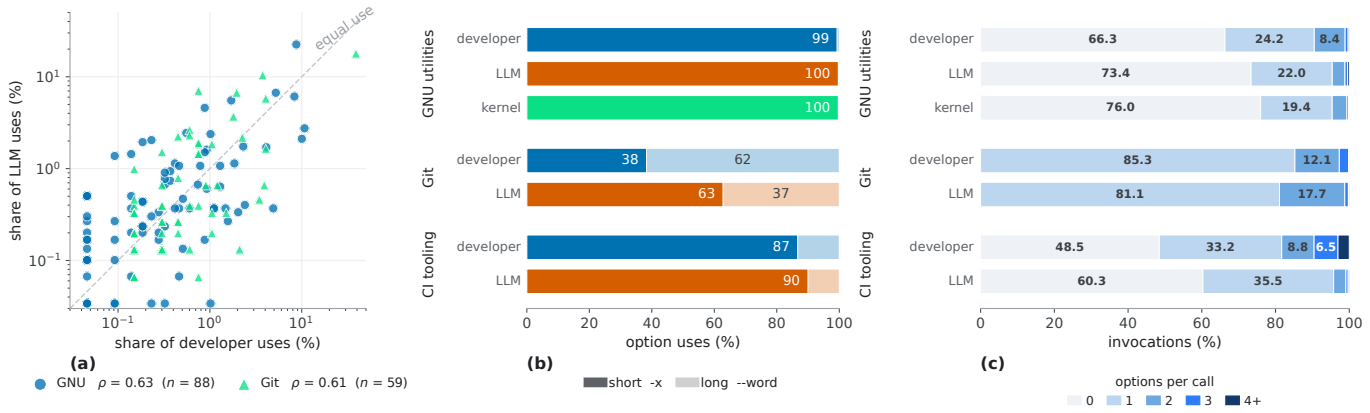


Fig. 3. Style of option use (RQ3): (a) popularity of shared options, (b) short vs. long spelling, (c) options per invocation (Git: only invocations with options).

diagonal, but with a large spread. The Spearman rank correlation is 0.63 over the 88 shared GNU options ($p < 10^{-10}$) and 0.61 over the 59 shared Git options ($p < 10^{-6}$), a moderate positive correlation. We exclude CI tooling, whose three shared options are too few for a rank test (*cf.* Figure 2(b)). In other words, the options that developers use most tend to be the ones that LLMs use most, but the rankings are far from identical. For example, `git commit -m` accounts for a much larger share of the LLM uses than of the developer uses (Table II).

For *spelling*, Figure 3(b) considers only options with both a short and a long form. On GNU, both corpora almost always use the short form (*e.g.*, `ls -l`), in 99% of the developer uses and in nearly all LLM uses. On Git, developers prefer the long form (62%, *e.g.*, `git commit --amend`), whereas LLMs prefer the short form (63%). On CI tooling, both prefer the short form, at 87% and 90%. Over all aliased options, the two corpora prefer the same form in 81 of 88 cases, and most disagreements concern Git. The two corpora thus differ more in which options they use than in how they spell them.

The *configuration depth*, that is, the number of options per invocation, is shown in Figure 3(c) for all invocations of each corpus. In both corpora, most invocations carry zero or one option. On GNU, 9.5% of the developer invocations carry two or more options, against 4.6% of the LLM invocations. On Git, where only invocations with options are recorded, the shares are 14.7% and 18.9%. CI tooling shows the largest difference. There, 18.3% of the developer invocations carry two or more options and 3.1% carry four or more, against 4.2% and 0.3% for LLMs. Since both corpora reach about the same number of CI options (RQ2), this is not a difference in which options are used. Rather, the CI steps generated by LLMs tend to consist of simpler invocations, whereas developers more often combine several options in a single call.

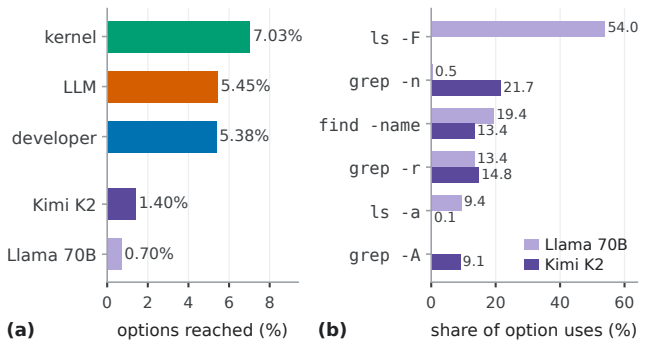


Fig. 4. Reach and most used options of the two LLM agents (RQ4).

Finding 3. LLMs and developers use the options they share in a broadly similar way. Their popularity rankings are moderately correlated (Spearman ρ of 0.63 on GNU and 0.61 on Git), and both prefer the same spelling for most aliased options (81 of 88, 92%). Both mostly pass at most one option per call, and the largest difference is on CI tooling, where developers combine two or more options more often (18.3% versus 4.2% of the invocations).

D. RQ4: Do Executing Agents Use More Options?

In RQ1 to RQ3, we studied commands that LLMs wrote into scripts and workflows. An agent that runs commands itself and sees their output could use more options, for instance by trying further options when a command fails. We therefore decided to compare two agents with the static corpora on the GNU programs (Figure 4), namely SWE- and DeepSWE-agent, each observed in 600 runs on GitHub issues (*cf.* Section III-C).

As a result, we observed that both agents reach fewer options than any static corpus. Llama 70B uses 19 distinct documented options, 0.70% of the GNU option set, and Kimi K2 uses 38 options (1.40%), although it issued 14,370 invocations, more than the static LLM corpus (8,795). The agents pass options more often than the static corpora, as only 29.4% (Llama 70B) and 28.2% (Kimi K2) of their invocations carry no option, against 66% to 76% in the static corpora. Their option uses are, however, highly concentrated (Figure 4(b)). For Llama 70B, `ls -F` alone accounts for 54% of the uses. For Kimi K2, `grep -n`, `grep -r`, and `find -name` together account for about half of the uses. The two agents have different profiles, but both are narrow.

The scaffolds of the agents explain much of this concentration. SWE-agent gives the model its own commands for opening, editing, and searching files, which make up two thirds of the actions in our sample, and its search commands call `find` and `grep` internally. The demonstration in its default configuration writes "but we should check to be sure" and then looks for a file with `ls -F`. The model repeats this sentence in 32 of the 600 runs, 21 times followed by `ls -F`. The prompt of DeepSWE-Agent tells the model to use `grep` instead of its search tool and to "Use grep with `-A -B -C` flags", and its search tool is described as running `grep -n`. The most frequent options of both agents are thus the options that their prompts name or demonstrate. Our agent results thus reflect the scaffolds as much as the models, and they do not show which options Llama 70B or Kimi K2 would choose on their own. They do show, however, that the options an LLM sees in its context strongly shape the options it uses. Moreover, the agents resolve GitHub issues, a task that consists mostly of navigating and searching code, whereas the static corpora contain scripts and workflows for many other tasks. A difference between the agents and the static corpora (*i.e.*, when LLMs were used as codding assistants) may thus come from the task and not only from executing commands.

Finding 4. Executing commands did not widen the options that LLM agents use. Both agents use fewer options than the static corpora (0.70% and 1.40% of the documented options, against at least 5.38%), mostly those that their prompts name or demonstrate, so the options in the context of an LLM strongly shape the options it uses.

V. Discussion

LLMs reproduce the option usage of developers rather than extend it. They use a similarly small share of the option sets, their rankings of shared options correlate moderately, and they spell these options alike. The options that only LLMs use are well known general purpose options, such as `grep -r` and `du -h`, not rare options that only the documentation describes. Our data cannot show why, but this fits LLMs that draw mainly on usage rather than on documentation. It may also explain why LLMs miss the options that only developers use, which tend to serve specific tasks (Table II). A rare option appears when a task has a concrete requirement, as when a developer installs a file for a given user with `install -o`, and the prompts behind generated scripts may rarely state such requirements. Generated commands are thus typical, as they suit routine tasks but can omit the options that a specific requirement needs, which functional tests rarely reveal.

Our results have three implications. First, generated commands should be checked against the requirements of their task, not only for whether they run. Benchmarks for command generation, such as NL2Bash [42] and Terminal-Bench [43], could include tasks whose requirements need rare options, and reach can measure how much of an option set a model covers. Second, option usage may narrow further. As generated scripts are committed and become training data, future models may concentrate even more on frequent options, since models trained on generated data can lose the rare parts of the original distribution [44]. Third, the context of an LLM is the most promising lever. The agents of RQ4 mostly used the options that their prompts named, usage examples in the prompt raised the accuracy of Claude on complex tool parameters from 72% to 90% [30], and relevant documentation improves the use of rare APIs [26]. Programs should therefore expose their configuration options to LLMs at generation time, as we

propose next. Whether this widens the options that LLMs use remains to be tested in future controlled experiments.

VI. Configuration Manifests and Contracts

Pretty much all software programs describe their options for human readers. Then, the `--help` text of a program lists its options, but option dependencies, mutual exclusions, deprecations, and security notes are often found only in manual pages, changelogs, or community knowledge, and some of this information is newer than the training data of an LLM. If exposing program options to LLMs at generation time helps, as the observations above suggest, programs need to provide their options in a form that tools can query. OpenCLI already proposes a description of command-line interfaces modelled on OpenAPI [45], and the Agent CLI Guidelines require tools to print a schema of their commands and flags, with types and defaults [46]. Neither describes the constraints among options or the implementation that a description belongs to. We outline two complementary mechanisms that add them.

A. Configuration Manifests

Next to its `--help` text, a program could publish a *configuration manifest* that describes its options in a form tools can read. The manifest would declare each option with its type, valid values, conflicts, and requirements, similar to an OpenAPI specification of a web API. A program could deliver the same manifest in two ways, through a dedicated flag, *e.g.*, `--llm`, which always matches the installed version, or as a `variability.md` file in its source repository, which can be read without installing the program. Such a file resembles the `llms.txt` files that websites publish for LLMs [47], but it would describe the options as a variability model, *e.g.*, as a *feature model* [1], which tools can analyse automatically [48], not only as prose. A tool could then check a generated command against the manifest, or give the LLM only the part of the manifest that is relevant to a task, so that long option lists do not overload its context. The Model Context Protocol (MCP) [49] already defines how LLMs discover tools and their schemas at runtime, and programs with a manifest could be offered as MCP tools. With usage data such as ours, manifests would also show which rarely used options could be removed or moved into separate, more specialised programs [33].

B. Configuration Contracts

A manifest describes one version of one program. The options of a program are added, deprecated, and removed between versions, and an LLM trained on data from 2023 cannot know about a deprecation in 2025. A `variability.md` file may also describe another version than the one installed, and a manifest does not identify the implementation. For example, GNU coreutils, BusyBox, and Toybox all provide an `ls` program, but with different option sets, so an option that works with one may be rejected by another. A *configuration contract* would bind a manifest to a version and an implementation, so that a tool can check whether a command is valid for the program that will run it, like the API contracts between services. The idea also applies beyond command-line options, to options set at compile time [14], environment variables, and configuration files.

Both mechanisms shift knowledge about program options from the training data of an LLM to information that the program itself should provide at generation time.

VII. Threats to Validity

Construct Validity: Our ground truth comes from the --help output of one version of each program, so undocumented options are missing and our reach values are likely upper bounds. Counting each pair of short and long spellings as one option raises reach (*e.g.*, from 5.45% to 7.54% for the GNU LLM corpus), but it does not change the comparison between corpora. Then, our extraction is lexical and works line by line, so it misses commands in quoted strings, such as those passed to `sh -c`, and commands built at runtime, and it may count text in here documents (`<<EOF`) as invocations. The CI extractor is simpler and discards many candidates, so the CI reach is a lower bound (Section III-E). Finally, a provenance marker shows that an LLM was involved, but not that developers left the artifact unchanged.

Internal Validity: The developer and LLM corpora come from different repositories and periods, so their differences may reflect tasks, project maturity, or program mix rather than the type of caller. We reduce this threat with three datasets that differ in domain and type of artifact. We also collected about the same number of developer and LLM files for GNU and CI tooling (Table I), but we could not control how many invocations these files contain, which gives the larger LLM corpora more opportunities to use rare options. For example, 60 of the 210 LLM workflows invoke `docker`, `npm`, or `pip`, against 31 of the 210 developer workflows, which use more prebuilt actions. The kernel corpus comes from a single project with its own tasks, so we treat it as a reference for one domain, not as a measure of expertise, and the agent results describe the scaffolds as much as the models (Section IV-D).

External Validity: Scripts with an explicit LLM provenance marker are still rare, so our corpora are modest in size. To check whether larger corpora would reach many more options, we drew 500 random samples of 90% of the files of each corpus. On GNU and Git, these samples already reach 95% to 96% of the options of the full corpus on average, so additional files would mostly repeat options that are already used. Kimi K2 points the same way, as it reaches fewer options than the static LLM corpus although it issued more invocations. We therefore do not expect larger corpora to change our conclusions. We study popular programs, the most favourable case for LLMs, and the models available when the artifacts were created, so we expect that less common programs and newer models may show different results. Finally, our corpora are samples, and the artifacts with a provenance marker may not represent all kinds of generated code.

VIII. Related Work

Configuration in Software Engineering: A large body of work studies software configuration as a source of complexity

and failures. Xu *et al.* showed that configuration spaces keep growing while most options are set by very few users [5]. Other work identifies the option responsible for a given behaviour [50], models how options affect performance [15], and mines the constraints that make a configuration valid [16]. Tërnava *et al.* measured the variability of 108 GNU coreutils programs and found that, for the 20 smallest of them, it grows across releases [17]. That most options of a mature system are rarely used is therefore well known. These studies, however, concern how people configure software. Closer to our work, Lian *et al.* found that LLMs used as configuration validators are biased towards popular configuration parameters [24]. Whereas they study LLMs that check configurations, we study LLMs as callers that choose configuration options of programs, and we use the option usage of developers as the baseline that shows whether a narrow reach is specific to LLMs.

Variability and Product Lines: The software product line community has already formalised the configuration space of a system as a set of features with constraints among them [2], [1], [51]. Several of their works measured how much of a configuration space tests exercise [52], studied how configuration options are structured and accessed in practice [53], and showed how far the space that a system declares can drift from the space its code implements [54]. Just like these studies, we separate the documented space, here the `--help` output of a program, from the used space, but we compare how two kinds of callers, LLMs and developers, use it.

Evaluation of Code Generated by LLMs: Most evaluations of code generation check functional correctness on curated benchmarks [55], [12] or on real repository issues [13]. Other work studies how developers use these tools [56], [6], [57], and shows that generated code can contain vulnerabilities [27], [58] or refer to packages that do not exist [28]. Mining studies have started to trace LLM use in repositories, through developers' conversations with ChatGPT [59], mentions of ChatGPT in commits and pull requests [7], statements in which developers acknowledge their use of generative AI [38], and pull requests opened by coding agents [8]. These efforts ask whether generated code works, whether it is safe, how developers use it, or how it enters a project, whereas we ask which options of the called programs it uses. The work on package hallucination is closest to ours, but we study options that exist and are documented, not names that do not exist.

Tool Use and the Command Line: Several works on LLMs that use tools studies how a model calls external functions and APIs [60], [61], [62]. These approaches assume a fixed catalogue of tools described by a schema, now standardised by the MCP [49], and even then LLMs hallucinate API calls or pass inaccurate arguments, which retrieving the relevant documentation reduces [63]. In this setting, retrieving the relevant documentation improves the generation of Bash commands [29], and documentation alone lets LLMs use a renamed command-line interface that they have never seen [32]. The command line has no common schema, and the implementation behind a program name can differ between systems, so an LLM knows in advance neither which implementation will run nor which options it accepts. Agent systems give the LLM direct access to a shell [10], and the command line is a target of natural language interfaces [42], [64], or score whether hard terminal tasks are solved [43]. All of this work treats the command as the output to produce or the action to run, whereas we treat it as the object of study and use developers' usage as the reference. To our knowledge, no prior work has measured how LLMs use the configuration spaces of the command-line programs they call.

IX. Conclusion

We studied how far LLMs reach into the documented option sets of 403 command-line programs and subcommands, through more than 31,000 invocations mined from public repositories. Between 90.1% and 98.0% of the documented options are never used in any of our corpora. Scripts attributed to LLMs reach about as far as those of developers, rank the options they share in a moderately similar order, and spell them alike, but the options that only LLMs use tend to be general purpose, whereas those that only developers use tend to serve a specific task. The Linux kernel scripts reach further, and two LLM agents that execute commands reach even fewer options, mostly those that their prompts name or demonstrate.

Our data cannot show why, but these results are consistent with LLMs following how program options are used in existing code rather than how they are documented. Rarely used or recently added options, and the options of internal or new programs that no public code calls, are then the hardest cases for LLMs, and functional tests are unlikely to reveal that they are missing. We therefore propose *configuration manifests*, through which programs expose their configuration options and the constraints among them to LLMs at generation time, and *configuration contracts*, that bind a manifest to a program version and an implementation. Our dataset and metrics provide a baseline for evaluating such interventions.

Future Work: Our next step is to define a concrete format for *configuration manifests* and *contracts*, *e.g.*, based on feature models [1] in the software product line literature, and to build them for programs with several versions and implementations from their help output and documentation. A controlled experiment can then give an LLM the manifest of a program, in full or selected for the task, and measure whether its reach and the validity of its commands change. Further directions include tasks that require rare or recently added options, newer models, and more agents and prompts.

Data Availability

The corpora, ground truth, scripts, and results of this study are available in our anonymised replication package at https://github.com/anonconfs/llms-miss.

Acknowledgment

Claude Opus 4.6 and 5.5 assisted mostly in writing the analysis scripts and charts, all of which the authors verified.

References

[1] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak, and A. S. Peterson, "Feature-oriented domain analysis (FODA) feasibility study," Software Engineering Institute, Carnegie Mellon University, Tech. Rep. CMU/SEI-90-TR-021, 1990. https://insights.sei.cmu.edu/library/feature-oriented-domain-analysis-foda-feasibility-study/

[2] S. Apel, D. Batory, C. Kästner, and G. Saake, *Feature-Oriented Software Product Lines: Concepts and Implementation*. Springer, 2013. https://doi.org/10.1007/978-3-642-37521-7

[3] T. Berger, R. Rublack, D. Nair, J. M. Atlee, M. Becker, K. Czarnecki, and A. Wąsowski, "A survey of variability modeling in industrial practice," in *Proceedings of the 7th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, 2013, pp. 7:1–7:8. https://doi.org/10.1145/2430502.2430513

[4] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, "Software configuration engineering in practice: Interviews, survey, and systematic literature review," *IEEE Transactions on Software Engineering*, vol. 46, no. 6, pp. 646–673, 2020. https://doi.org/10.1109/TSE.2018.2867847

[5] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadker, "Hey, you have given me too many knobs! Understanding and dealing with over-designed configuration in system software," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 307–319. https://doi.org/10.1145/2786805.2786852

[6] J. T. Liang, C. Yang, and B. A. Myers, "A large-scale survey on the usability of AI programming assistants: Successes and challenges," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13. https://doi.org/10.1145/3597503.3608128

[7] R. Tufano, A. Mastropaolo, F. Pepe, O. Dabić, M. Di Penta, and G. Bavota, "Unveiling ChatGPT's usage in open source projects: A mining-based study," in *Proceedings of the 21st International Conference on Mining Software Repositories (MSR)*. ACM, 2024, pp. 571–583. https://doi.org/10.1145/3643991.3644918

[8] H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI coding agents on GitHub," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. ACM, 2026, pp. 1029–1033. https://doi.org/10.1145/3793302.3797249

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. https://arxiv.org/abs/2405.15793

[10] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "OpenHands: An open platform for AI software developers as generalist agents," in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. https://arxiv.org/abs/2407.16741

[11] Reproducible Builds Project, "Archive metadata: Creating reproducible tar archives," https://reproducible-builds.org/docs/archives/, 2024.

[12] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 21 558–21 572. https://doi.org/10.52202/075280-0943

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2310.06770

[14] X. Tërnava, M. Acher, L. Lesoil, A. Blouin, and J.-M. Jézéquel, "Scratching the surface of ./configure: Learning the effects of compile-time options on binary size and gadgets," in *Proceedings of the 20th International Conference on Software and Systems Reuse (ICSR)*. Springer, 2022, pp. 41–58. https://doi.org/10.1007/978-3-031-08129-3_3

[15] N. Siegmund, A. Grebhahn, S. Apel, and C. Kästner, "Performance-influence models for highly configurable systems," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 284–294. https://doi.org/10.1145/2786805.2786845

[16] S. Nadi, T. Berger, C. Kästner, and K. Czarnecki, "Mining configuration constraints: Static analyses and empirical results," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 140–151. https://doi.org/10.1145/2568225.2568283

[17] X. Tërnava, G. A. Randrianaina, L. Lesoil, and M. Acher, "Small yet configurable: Unveiling null variability in software," arXiv preprint arXiv:2604.15957, 2026. https://arxiv.org/abs/2604.15957

[18] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2011, pp. 159–172. https://doi.org/10.1145/2043556.2043572

[19] H. S. Gunawi, M. Hao, T. Leesatapornwongsa, T. Patana-anake, T. Do, J. Adityatama, K. J. Eliazar, A. Laksono, J. F. Lukman, V. Martin, and A. D. Satria, "What bugs live in the cloud? A study of 3000+ issues in cloud systems," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. ACM, 2014, pp. 7:1–7:14. https://doi.org/10.1145/2670979.2670986

[20] T. Xu and Y. Zhou, "Systems approaches to tackling configuration errors: A survey," *ACM Computing Surveys*, vol. 47, no. 4, pp. 70:1–70:41, 2015. https://doi.org/10.1145/2791577

[21] Anthropic, "Claude code: Best practices for agentic coding," https://www.anthropic.com/engineering/claude-code-best-practices, 2025.

[22] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large language models struggle to learn long-tail knowledge," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. https://arxiv.org/abs/2211.08411

[23] A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang, "Why language models hallucinate," arXiv preprint arXiv:2509.04664, 2025. https://arxiv.org/abs/2509.04664

[24] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, "Large language models as configuration validators," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025, pp. 1704–1716. https://doi.org/10.1109/ICSE55347.2025.00017

[25] L. Twist, M. Harman, D. Syme, J. Noppen, H. Yannakoudakis, D. Nauck, and J. M. Zhang, "A study of LLMs' preferences for libraries and programming languages," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. https://arxiv.org/abs/2503.17181

[26] N. Jain, R. Kwiatkowski, B. Ray, M. K. Ramanathan, and V. Kumar, "On mitigating code LLM hallucinations with API documentation," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 237–248. https://doi.org/10.1109/ICSE-SEIP66354.2025.00027

[27] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of github copilot's code contributions," in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768. https://doi.org/10.1109/SP46214.2022.9833571

[28] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala, "We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security)*, 2025, pp. 3687–3706. https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen

[29] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "DocPrompting: Generating code by retrieving the docs," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2207.05987

[30] Anthropic, "Introducing advanced tool use on the Claude developer platform," https://www.anthropic.com/engineering/advanced-tool-use, 2025.

[31] C. Wang, K. Huang, J. Zhang, Y. Feng, L. Zhang, Y. Liu, and X. Peng, "LLMs meet library evolution: Evaluating deprecated API usage in LLM-based code completion," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. https://arxiv.org/abs/2406.09834

[32] C.-Y. Hsieh, S.-A. Chen, C.-L. Li, Y. Fujii, A. Ratner, C.-Y. Lee, R. Krishna, and T. Pfister, "Tool documentation enables zero-shot tool-usage with large language models," arXiv preprint arXiv:2308.00675, 2023. https://arxiv.org/abs/2308.00675

[33] G. J. Holzmann, "Code inflation," *IEEE Software*, vol. 32, no. 2, pp. 10–13, 2015.

[34] V. R. Basili, G. Caldiera, and H. D. Rombach, "The goal question metric approach," in *Encyclopedia of Software Engineering*. John Wiley & Sons, 1994.

[35] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 426–437. https://doi.org/10.1145/2970276.2970358

[36] Nebius, "SWE-agent trajectories," Hugging Face Dataset, https://huggingface.co/datasets/nebius/SWE-agent-trajectories, 2024.

[37] SWE-Factory, "DeepSWE-Agent trajectories with Kimi K2," Hugging Face Dataset, https://huggingface.co/datasets/SWE-Factory/DeepSWE-Agent-Kimi-K2-Trajectories-2.8K, 2025.

[38] T. Xiao, Y. Fan, F. Calefato, C. Treude, R. G. Kula, H. Hata, and S. Baltes, "Self-admitted GenAI usage in open-source software," *IEEE Transactions on Software Engineering*, vol. 52, no. 6, pp. 1891–1910, 2026. https://doi.org/10.1109/TSE.2026.3681886

[39] C. Gini, "Measurement of inequality of incomes," *The Economic Journal*, vol. 31, no. 121, pp. 124–126, 1921. https://doi.org/10.2307/2223319

[40] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

[41] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904. https://doi.org/10.2307/1412159

[42] X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst, "NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system," in *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC)*, 2018. https://aclanthology.org/L18-1491/

[43] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj *et al.*, "Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces," in *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026, pp. 40 903–40 986. https://proceedings.iclr.cc/paper_files/paper/2026/hash/444a3737adaee10d86ad2ef5f74468e6-Abstract-Conference.html

[44] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal, "AI models collapse when trained on recursively generated data," *Nature*, vol. 631, no. 8022, pp. 755–759, 2024. https://doi.org/10.1038/s41586-024-07566-y

[45] P. Svensson and B. Lail, "OpenCLI specification, version 0.1," https://opencli.org/, 2025.

[46] Agent CLI Guidelines contributors, "Agent CLI guidelines, version 0.4," https://aclig.dev/, 2026.

[47] J. Howard, "The /llms.txt file," https://llmstxt.org/, 2024.

[48] D. Benavides, S. Segura, and A. Ruiz-Cortés, "Automated analysis of feature models 20 years later: A literature review," *Information Systems*, vol. 35, no. 6, pp. 615–636, 2010. https://doi.org/10.1016/j.is.2010.01.001

[49] Anthropic, "Introducing the Model Context Protocol," Anthropic Announcement, 2024. https://www.anthropic.com/news/model-context-protocol

[50] S. Zhang and M. D. Ernst, "Which configuration option should i change?" in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. ACM, 2014, pp. 152–163. https://doi.org/10.1145/2568225.2568251

[51] D. Batory, "Feature models, grammars, and propositional formulas," in *Proceedings of the 9th International Conference on Software Product Lines (SPLC)*, ser. LNCS, vol. 3714. Springer, 2005, pp. 7–20. https://doi.org/10.1007/11554844_3

[52] A. Halin, A. Nuttinck, M. Acher, X. Devroey, G. Perrouin, and B. Baudry, "Test them all, is it worth it? Assessing configuration sampling on the jhipster web development stack," *Empirical Software Engineering*, vol. 24, no. 2, pp. 674–717, 2019. https://doi.org/10.1007/s10664-018-9635-4

[53] D. Jin, X. Qu, M. B. Cohen, and B. Robinson, "Configurations everywhere: Implications for testing and debugging in practice," in *Companion Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 215–224. https://doi.org/10.1145/2591062.2591191

[54] R. Tartler, D. Lohmann, J. Sincero, and W. Schröder-Preikschat, "Feature consistency in compile-time-configurable system software: Facing the Linux 10,000 feature problem," in *Proceedings of the 6th European Conference on Computer Systems (EuroSys)*. ACM, 2011, pp. 47–60. https://doi.org/10.1145/1966445.1966451

[55] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. https://doi.org/10.48550/arXiv.2107.03374

[56] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proceedings of the ACM on Programming Languages (PACMPL)*, vol. 7, no. OOPSLA1, pp. 85–111, 2023. https://doi.org/10.1145/3586030

[57] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "Reading between the lines: Modeling user behavior and costs in AI-assisted programming," in *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*. ACM, 2024, pp. 1–16. https://doi.org/10.1145/3613904.3641936

[58] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2023, pp. 2785–2799. https://doi.org/10.1145/3576915.3623157

[59] T. Xiao, C. Treude, H. Hata, and K. Matsumoto, "DevGPT: Studying developer-ChatGPT conversations," in *Proceedings of the 21st International Conference on Mining Software Repositories (MSR)*. ACM, 2024, pp. 227–230. https://doi.org/10.1145/3643991.3648400

[60] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 68 539–68 551. https://doi.org/10.52202/075280-2997

[61] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2307.16789

[62] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/pdf/2210.03629

[63] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 126 544–126 565. https://doi.org/10.52202/079017-4020

[64] F. Westenfelder, E. Hemberg, S. Moskal, U.-M. O'Reilly, and S. Chiricescu, "LLM-supported natural language to Bash translation," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2025, pp. 11 135–11 147. https://doi.org/10.18653/v1/2025.naacl-long.555